

Complete Analysis of GenAI code

Name of the code files: guess_genned_chatgpt.c

guess_genned_claude.c

Corrected_Claude_genned_code.c

Corrected_Claude_genned_code (Linux executable file)

How they were accessed: ChatGPT, Claude

Prompt used for generating code:

Use two GenAI's to generate C code to implement the following requirements: Objective
Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number.

Requirements

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Code Generated:

ChatGPT:

```
</> C

#include <stdio.h>

int main() {
    int secretNumber = 7;
    int guess;
    int tries = 0;
    int correct = 0;

    while (tries < 3) {
        printf("Guess the secret number (1-10): ");
        scanf("%d", &guess);

        tries++;

        if (guess == secretNumber) {
            printf("Correct! You guessed the secret number!\n");
            correct = 1;
            break;
        } else if (guess < secretNumber) {
            printf("Too low!\n");
        } else {
            printf("Too high!\n");
        }
    }

    if (correct) {
        printf("You win!\n");
    } else {
        printf("You lose! The secret number was %d.\n", secretNumber);
    }

    return 0;
}
```

Claude:

```
#include <stdio.h>

int main() {
    int secret = 7;
    int guess;
    int attempts = 3;
    int won = 0;

    printf("Guess the secret number (between 1 and 10). You have %d tries.\n", attempts);

    for (int i = 1; i <= attempts; i++) {
        printf("Attempt %d: Enter your guess: ", i);
        scanf("%d", &guess);

        if (guess == secret) {
            won = 1;
            break;
        } else if (guess < secret) {
            printf("Too low!\n");
        } else {
            printf("Too high!\n");
        }
    }

    if (won) {
        printf("Congratulations! You guessed the secret number %d correctly!\n", secret);
    } else {
        printf("Sorry, you're out of tries. The secret number was %d.\n", secret);
    }

    return 0;
}
```

Program Analysis for both codes:

Correctness: Both codes can handle the sample output for valid input. However, neither code is particularly built to handle edge cases or handle errors particularly well.

Time complexity: Both codes have time complexity of $O(1)$. This is because despite the for or while loop, the code runs only a constant number of times, so it is always fixed.

Space complexity: The program only uses a few int variables that do not dynamically increase in size, like how an array does. Hence, the space complexity of both codes is also $O(1)$. Both use the same number of constant variables which is 4. However, the Claude code pulls slightly ahead as a for loop (Claude) translates to less memory compared to a while loop.

Execution time: Since the code is dependent on the user's input, the execution time will vary for both code.

Maintainability: Neither code has good maintainability due to the lack of comments. However, the Claude code slightly has the edge as its statements are slightly more explicit in nature, as shown by having the attempt number written for everytime the code is run.

Which code is better?

Despite both codes being very similar, the Claude code would be the one I would prefer using. Since it uses a for loop, which uses less memory compared to a while loop used by ChatGPT, which helps its space complexity. Similarly, it is better for execution time and time complexity as a for loop is more efficient than a while loop. As mentioned above, it also has slightly better maintainability due to its more explicit prints.

How the code was improved:

Every line was commented on for better readability and maintainability, which allows the user to know exactly what's happening in the code. The prints now give out meaningful error messages rather than just plain invalid input to show what was the error. A range validation was added as the original accepted any integer valid or invalid. Now invalid out of range guesses go through another loop and release an error message. Furthermore, error handling was added, the program can now handle EOF errors separately and exits without any issues. A counter of `i` - - was also added to only begin the count when legitimate input should be added.

Corrected Code

```
#include <stdio.h>

int main() {
    int secret = 7; // the number the player is trying to guess
    int guess;      // stores the player's current guess each round
    int attempts = 3; // how many guesses the player gets
    int won = 0;     // flag: 0 = hasn't won yet, 1 = guessed correctly
                    // (used after the loop to decide which final message to print)

    // Explain the rules to the player before the game starts
    printf("Guess the secret number (between 1 and 10). You have %d tries.\n", attempts);

    // Main game loop: runs at most "attempts" times.
    // "i" also doubles as the attempt number shown to the player.
    for (int i = 1; i <= attempts; i++) {
        printf("Attempt %d: Enter your guess: ", i);

        // scanf returns the number of items it successfully read.
        // We expect 1 (one integer). Anything else means the read failed.
        int result = scanf("%d", &guess);

        // --- Error case 1: input stream closed (e.g., Ctrl+D, piped file ran out) ---
        // Must be checked BEFORE the general "result != 1" check below,
```

```

// because EOF also satisfies "!= 1" and would otherwise cause
// an infinite loop (trying to re-read from a stream that's already closed).
if (result == EOF) {
    fprintf(stderr,
        "\nError: Input stream ended unexpectedly (EOF received). "
        "No more input is available to read. Exiting program.\n");
    return 1; // non-zero return signals abnormal termination to the OS/shell
}

// --- Error case 2: input wasn't a valid integer (e.g., letters, symbols) ---
if (result != 1) {
    fprintf(stderr,
        "Error: Attempt %d expects a whole number, but non-numeric input was entered. "
        "Please type digits only (e.g., 5), then press Enter.\n", i);

    // scanf leaves the bad characters sitting in the input buffer.
    // If we don't clear them, the next scanf call will immediately
    // fail again on the same leftover text. This loop discards
    // characters up to (and including) the next newline.
    while (getchar() != '\n' && getchar() != EOF);

    i--; // undo this iteration's increment so bad input
        // doesn't cost the player a real attempt
    continue; // skip the rest of this iteration, re-prompt
}

// --- Error case 3: valid integer, but outside the allowed range ---
if (guess < 1 || guess > 10) {
    fprintf(stderr,
        "Error: %d is outside the allowed range. "
        "Your guess must be between 1 and 10 (inclusive).\n",
        guess);
    i--; // same idea: don't penalize the player for invalid input
    continue;
}

// --- Normal game logic: compare guess to the secret number ---
if (guess == secret) {
    won = 1; // mark as won...
    break; // ...and exit the loop immediately (no need to keep guessing)
} else if (guess < secret) {
    printf("Too low! Try a higher number.\n");
} else {
    printf("Too high! Try a lower number.\n");
}
// If neither error case nor a correct guess, the loop simply
// continues to the next attempt (i increments normally).
}

```

```
// After the loop: report the final outcome based on the "won" flag.  
// This runs whether the loop ended via break (won) or ran out  
// of attempts (loop condition false).  
if (won) {  
    printf("Congratulations! You guessed the secret number %d correctly!\n", secret);  
} else {  
    printf("Sorry, you're out of tries. The secret number was %d.\n", secret);  
} //end of else loop  
  
return 0; // 0 signals successful/normal program termination  
} //end of the loop
```